

Lab 1: Link-layer

Userspace Ethernet II device management and frame I/O using libpcap. Implementation and checkpoint evidence were verified on the Linux host lab-comp-net. Evidence was recorded on October 9, 2026.

WT1 - Packet trace analysis

Filter: eth.src == 6a:15:0a:ba:9b:7c. The following answers concern the third frame in the filtered results, which is frame 12 in the original trace.

Question	Answer
Number of filtered frames	827
Destination address and significance	ff:ff:ff:ff:ff:ff is the Ethernet broadcast address. It allows the DHCP Discover message to reach hosts in the local broadcast domain, including DHCP servers.
Byte at offset 71 (zero-based)	0x15

Source arrangement

Source	Responsibility
src/device.h; src/device.c	PT1: registration, lookup, handle ownership, MAC discovery and caching, cleanup.
src/packetio.h; src/packetio.c	PT2: Ethernet II construction/injection, callback registration and synchronous capture.
src/test/list_devices.c	CP1: enumerate capture devices with pcap_findalldevs.
src/test/send_demo.c; receive_demo.c	CP2: send and receive a 0x88b5 frame with a known payload.
src/test/test_*.c	Device/MAC integration checks and mocked sending/receiving checks.

Build and test

Requires Linux, a C compiler, make, libpcap development headers and the libpcap library. All executable output is placed in ./tmp. In the development tree test programs are under test/; this submission places them under src/test/. The Makefile supports both layouts.

```
make
make check
make check-device DEVICE=enp3s0
```

make check needs no root privileges and sends no real traffic. make check-device invokes sudo for capture-device access; replace enp3s0 with an Ethernet interface present on the test host.

PT1 - Device management

`addDevice` validates the name and capacity, reuses an existing device ID on duplicate registration, and opens a live pcap handle. MAC discovery enumerates addresses and accepts a six-byte AF_PACKET / ARPHRD_ETHER address. The name, handle and copied MAC are stored before the registered-device count is increased.

`findDevice` returns the registered ID or -1. `getDeviceHandle` and `getDeviceMAC` reject invalid IDs; the MAC getter copies the cache into caller-owned storage. `clearDevices` closes all handles and resets registration. Failed MAC discovery closes the newly opened handle without publishing a device.

CP1 - Interface detection

Attachment: `checkpoints/cp1.typescript`. The record shows `ip -brief` link and the output of the supplied `list_devices` program. Physical interface `enp3s0`, virtual interfaces `lab1rx` and `lab1tx`, loopback `lo`, and `libpcap` pseudo-devices are detected.

```
Device 0: enp3s0
Network: 10.129.80.0
Subnet mask: 255.255.252.0
Device 1: lab1rx
Device 2: lab1tx
Device 3: any
Device 4: lo
```

The beginning of the original record contains a missing-executable error. The program was then compiled and rerun successfully in the same session. The raw record is retained unchanged. Messages about missing IPv4 information on veth or pseudo-devices do not mean enumeration failed: the interface names were already detected, and the frame test does not require IPv4 configuration.

PT2 - Frame I/O

`sendFrame` accepts a payload of 0-1500 bytes, a destination MAC, EtherType and registered device ID. It constructs a 14-byte header, stores EtherType in network byte order, zero-pads frames to at least 60 bytes excluding FCS, and checks the byte count returned by `pcap_inject`. Buffer allocation and error paths are checked and released.

`setFrameReceiveCallback` registers or clears the callback. `receiveFrames` checks the device link type, waits synchronously for a positive number of complete frames, skips short/truncated captures, and passes `caplen` and device ID to the callback. Capture and callback failures return -1 with diagnostics and `errno`. Device cleanup remains owned by the device module.

Validation and scope

The build uses `-Wall -Wextra -Wpedantic -Werror`. All 19 send-frame checks and 24 receive/callback checks pass, as do the device and MAC integration checks. MAC bytes are independently compared with Linux sysfs; output bounds, copy ownership and open-descriptor cleanup are checked.

This Lab1 implementation targets ordinary untagged Ethernet II interfaces. It stores up to 16 devices, uses a synchronous per-device receive loop and a single callback registration, and assumes the MAC is unchanged after registration. It does not compute FCS, as permitted by the assignment. No named Lab1 task or interface is omitted; `not-implemented.pdf` is an empty file, following the handout instruction for a completed submission.

CP2 - Frame injection and capture

Attachments: checkpoints/cp2-send.typescript and checkpoints/cp2-receive.typescript. The recorded test uses a directly connected veth pair, both UP, with no IPv4 address required. The sender and receiver are both supplied programs using this implementation.

```
send_demo -> lab1tx ===== lab1rx -> receive_demo
              c6:71:04:ac:25:1c          0a:7e:59:41:3f:27
```

The sender injects EtherType 0x88b5 and the nine-byte payload lab1-test. The receiver installs an ether proto 0x88b5 filter, captures one complete frame, and prints it through the registered callback. Reception completed confirms the loop returned successfully.

Byte offsets	Field	Observed value
0-5	Destination MAC	0a:7e:59:41:3f:27
6-11	Source MAC	c6:71:04:ac:25:1c
12-13	EtherType	88 b5
14-22	Payload	6c 61 62 31 2d 74 65 73 74 = lab1-test
23-59	Padding	37 zero bytes

```
Received Ethernet frame: device=0, length=60 bytes
Source: c6:71:04:ac:25:1c
Destination: 0a:7e:59:41:3f:27
EtherType: 0x88b5
Reception completed.
```

Reproduce the checkpoint

After make, create the veth pair below if it does not already exist. Start the receiver in terminal A before sending from terminal B. Newly created interfaces may have different MAC addresses, so the sender reads the current peer address.

```
sudo ip link add lab1tx type veth peer name lab1rx
sudo ip link set lab1tx up
sudo ip link set lab1rx up

# Terminal A
sudo ./tmp/lab-netstack-receive-demo lab1rx

# Terminal B
peer_mac=$(cat /sys/class/net/lab1rx/address)
sudo ./tmp/lab-netstack-send-demo lab1tx "$peer_mac"

# After the demonstration
sudo ip link delete lab1tx
```

The receive record includes both interfaces and their MACs; the send record shows the matching destination and payload. The frame length, fields and content agree across the two records, demonstrating both libpcap injection and callback-based capture.